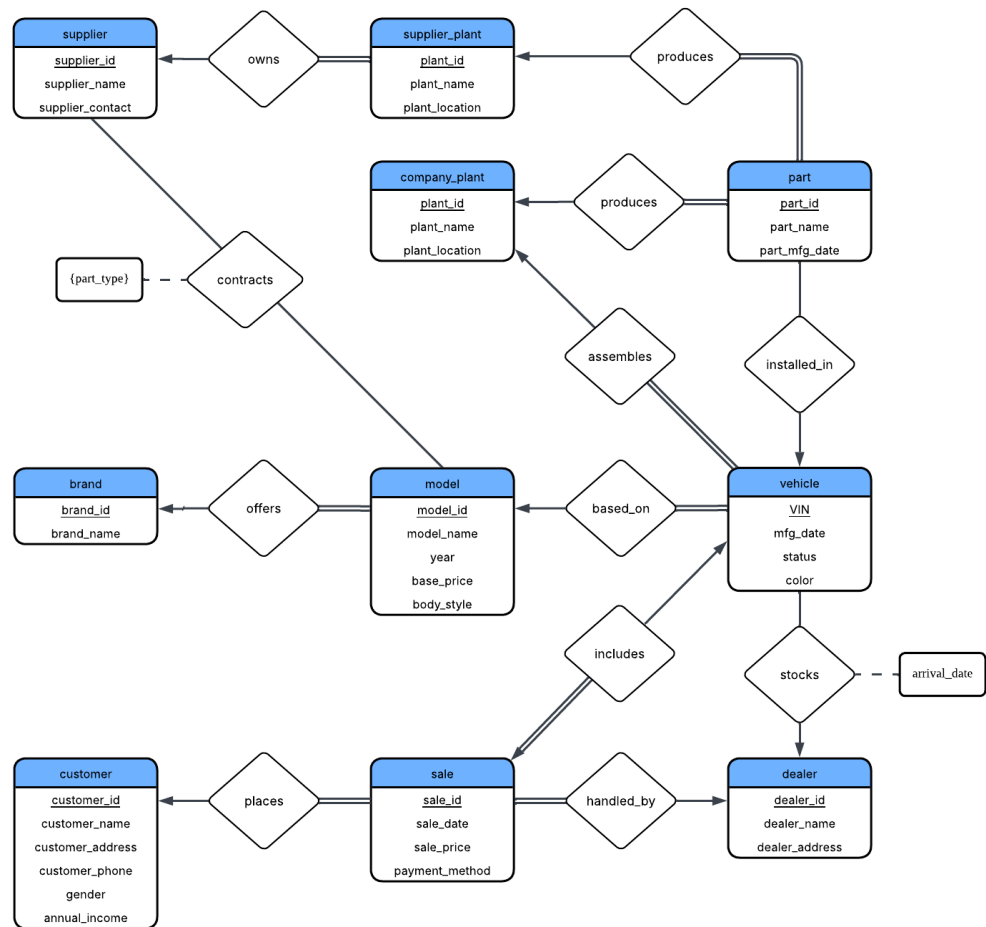


[Project2] Schema design & MySQL Implementation

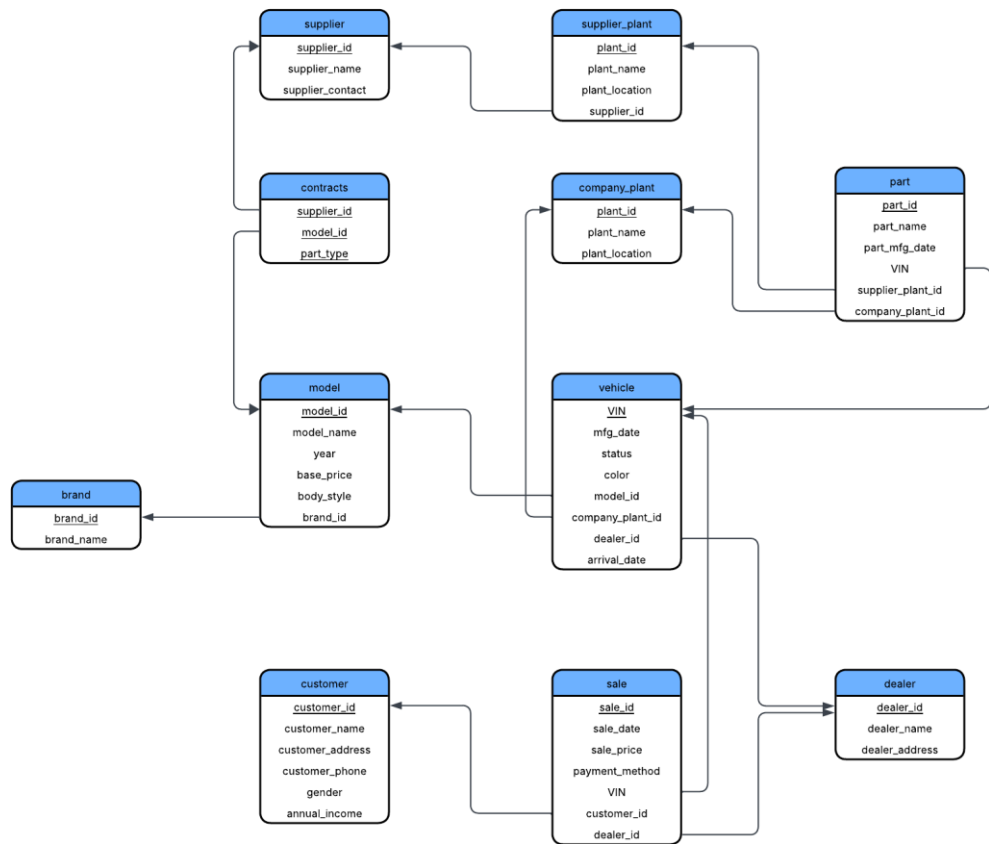
20210041 박민성

1. Logical Schema Design

1) E-R Diagram



2) Relational Schema Transformation



- 기존 ERD 에서 one-to-many relation 은 모두 many 쪽 relation 에 one 쪽의 PK 를 FK 로 추가해서 변환했습니다.
- Vehicle 과 sale 의 includes relation 에서, vehicle 에 sale_id 를 추가하면 아직 판매되지 않은 차량의 sale_id 가 NULL 이 되는 것을 피하기 위해 sale 쪽에 VIN 을 FK 로 추가했습니다.
- Supplier 와 model 의 contracts relation 은 many-to-many 이므로 contracts 를 생성한 후에 supplier 와 model 의 PK 인 supplier_id 와 model_id 를 FK 로 가져왔습니다. 기존 ERD 에서 한 공급업체가 한 모델에 공급하는 부품 종류를 나타내는 다중값 속성 {part_type}을 contracts 의 PK 로 설정해서 다중값 속성을 제거했습니다.

2. Normalization Analysis

- brand (brand_id, brand_name)

FD: brand_id → brand_name

Candidate key: {brand_id}

유일한 결정자 brand_id가 superkey이므로 BCNF를 만족합니다.

- model (model_id, model_name, year, base_price, body_style, brand_id)
FD: model_id → model_name, year, base_price, body_style, brand_id
Candidate key: {model_id}
유일한 결정자 model_id가 superkey이므로 BCNF를 만족합니다.
- supplier (supplier_id, supplier_name, supplier_contact)
FD: supplier_id → supplier_name, supplier_contact
Candidate key: {supplier_id}
유일한 결정자 supplier_id가 superkey이므로 BCNF를 만족합니다.
- supplier_plant (plant_id, plant_name, plant_location, supplier_id)
FD: plant_id → plant_name, plant_location, supplier_id
Candidate key: {plant_id}
유일한 결정자 plant_id가 superkey이므로 BCNF를 만족합니다.
- company_plant (plant_id, plant_name, plant_location)
FD: plant_id → plant_name, plant_location
Candidate key: {plant_id}
유일한 결정자 plant_id가 superkey이므로 BCNF를 만족합니다.
- part (part_id, part_name, part_mfg_date, VIN, supplier_plant_id, company_plant_id)
FD: part_id → part_name, part_mfg_date, VIN, supplier_plant_id, company_plant_id
Candidate key: {part_id}
유일한 결정자 part_id가 superkey이므로 BCNF를 만족합니다.
- vehicle (VIN, mfg_date, status, color, model_id, company_plant_id, dealer_id, arrival_date)
FD: VIN → mfg_date, status, color, model_id, company_plant_id, dealer_id, arrival_date
유일한 결정자 VIN이 superkey이므로 BCNF를 만족합니다.
- customer (customer_id, customer_name, customer_address, customer_phone, gender, annual_income)

FD: $\text{customer_id} \rightarrow \text{customer_name}, \text{customer_address}, \text{customer_phone}, \text{gender}, \text{annual_income}$

Candidate key: $\{\text{customer_id}\}$

유일한 결정자 customer_id 가 superkey이므로 BCNF를 만족합니다.

- dealer ($\text{dealer_id}, \text{dealer_name}, \text{dealer_address}$)

FD: $\text{dealer_id} \rightarrow \text{dealer_name}, \text{dealer_address}$

유일한 결정자 dealer_id 가 superkey이므로 BCNF를 만족합니다.

- sale ($\text{sale_id}, \text{sale_date}, \text{sale_price}, \text{payment_method}, \text{VIN}, \text{customer_id}, \text{dealer_id}$)

FD1: $\text{sale_id} \rightarrow \text{sale_date}, \text{sale_price}, \text{payment_method}, \text{VIN}, \text{customer_id}, \text{dealer_id}$

FD2: $\text{VIN} \rightarrow \text{sale_id}, \text{sale_date}, \text{sale_price}, \text{payment_method}, \text{customer_id}, \text{dealer_id}$

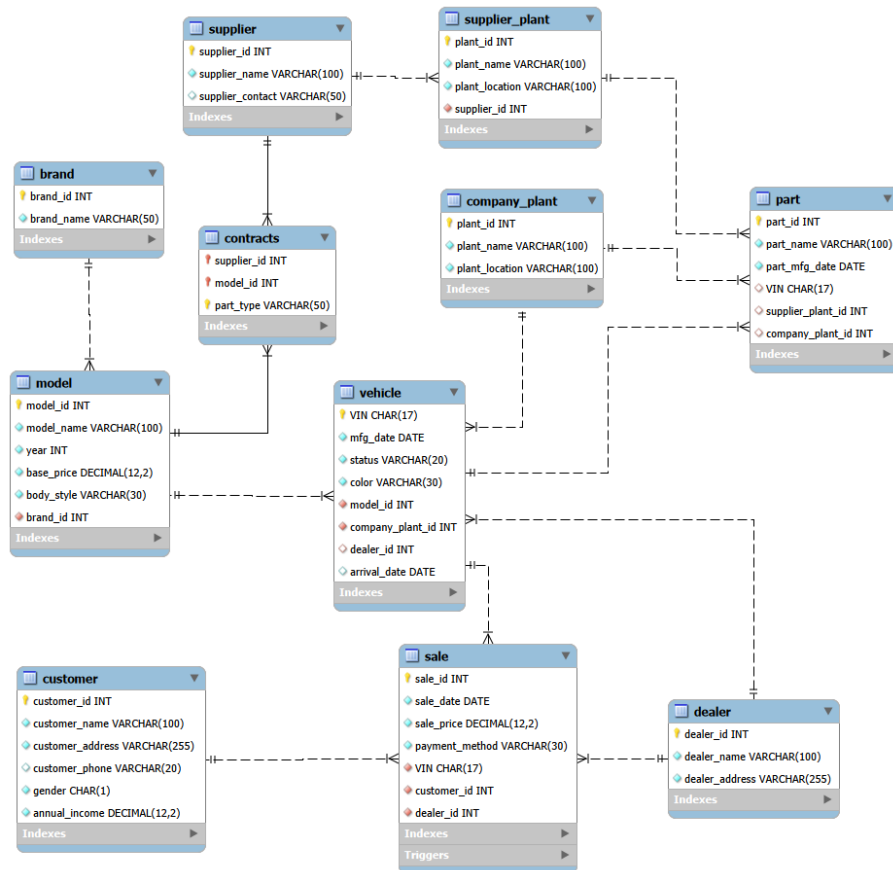
Candidate key: $\{\text{sale_id}\}, \{\text{VIN}\}$

두 결정자가 모두 superkey이므로 BCNF를 만족합니다.

- constracts ($\text{supplier_id}, \text{model_id}, \text{part_type}$)

Candidate key가 $\{\text{supplier_id}, \text{model_id}, \text{part_type}\}$ 인 all-key relation이므로 non-trivial FD가 존재하지 않아 BCNF를 만족합니다.

3. Physical Implementation



1) Data type selection rationale & Constraint implementation and business rule enforcement

- brand: brand_id를 INT 기본키로 설정했습니다. brand_name을 VARCHAR(50) NOT NULL로 설정했습니다.
- supplier: supplier_id를 INT 기본키로 설정했습니다. supplier_name을 VARCHAR(100) NOT NULL로 설정하고, supplier_contact는 연락처가 없을 수 있으므로 VARCHAR(50) NULL로 설정했습니다.
- company_plant: plant_id를 INT 기본키로 설정했습니다. plant_name과 plant_location을 각각 VARCHAR(100) NOT NULL로 설정했습니다.
- customer: customer_id를 INT 기본키로 설정했습니다. customer_name과 customer_address는 VARCHAR NOT NULL로, customer_phone은 선택 정보이므로 NULL로 설정했습니다. gender는 'M'/'F' 한 글자만 저장하는 것을 전제로 CHAR(1)로, annual_income은 금액 데이터이므로 부동소수점 오차를 피하기 위해 DECIMAL(12,2)로 정의했습니다. 이 두 속성은

Query 1의 성별, 소득 구간 분석에 직접 사용되므로 NOT NULL로 강제했습니다. 음수 소득을 막기 위해 CHECK(annual_income>=0) 제약을 부여했습니다.

- dealer: dealer_id를 INT 기본키로 설정했습니다. dealer_name과 dealer_address를 각각 VARCHAR NOT NULL로 설정했습니다.
- model: model_id를 INT 기본키로 설정했습니다. model_name, year을 NOT NULL로 설정했습니다. body_style의 경우 Query 5의 필터 기준이 되므로 반드시 값을 갖도록 NOT NULL로 강제했습니다. base_price는 금액이므로 DECIMAL(12,2)로 정의하고, 음수 가격을 막기 위해 CHECK(base_price>=0) 제약을 부여했습니다. offers 관계의 brand_id를 외래키로 두되, 브랜드 삭제 시 모델이 사라지지 않도록 ON DELETE RESTRICT, 식별자 변경 시 ON UPDATE CASCADE로 설정했습니다.
- supplier_plant: plant_id를 INT 기본키로 설정했습니다. plant_name, plant_location을 NOT NULL로 설정했습니다. 모든 공급업체 공장은 소속 공급업체가 반드시 존재하므로 supplier_id를 NOT NULL로 설정하고, owns 관계의 외래키이므로 ON DELETE RESTRICT, ON UPDATE CASCADE로 설정했습니다.
- vehicle: VIN은 17자리 표준 표현을 위해 고정 길이 CHAR(17)로 정의하고 기본키로 설정했습니다. mfg_date, status, color는 모든 차량이 갖는 값이므로 NOT NULL로 설정했습니다. stocks 관계의 dealer_id와 arrival_date는 아직 딜러에게 입고되지 않은 차량을 표현하기 위해 NULL을 허용했습니다. based_on의 model_id와 assemblies의 company_plant_id는 필수 외래키이므로 NOT NULL로 설정하고, ON DELETE RESTRICT, ON UPDATE CASCADE로 설정했습니다. stocks의 dealer_id는 딜러 삭제 시 ON DELETE SET NULL로 설정했습니다.
- part: part_id를 INT 기본키로 설정했습니다. part_mfg_date는 Query 2에 사용되는 값이므로 NOT NULL로 설정했습니다. installed_in 관계의 VIN은 아직 차량에 장착되지 않은 재고 부품을 표현하기 위해 CHAR(17) NULL로 설정했습니다. VIN은 차량 삭제 시 ON DELETE SET NULL로, supplier_plant_id, company_plant_id는 ON DELETE RESTRICT로 설정했습니다.

니다. 개별 부품은 공급업체 공장 또는 본사 공장 중 한 곳에서만 생산되므로, 두 외래키 중 정확히 하나만 값을 갖도록 CHECK 제약으로 강제했습니다.

- sale: sale_id를 INT 기본키로 설정했습니다. sale_date, payment_method를 NOT NULL로, sale_price는 금액이므로 DECIMAL(12,2) NOT NULL로 정의하고 CHECK(sale_price>=0) 제약을 부여했습니다. includes의 VIN, places의 customer_id, handled_by의 dealer_id를 외래키로 두고 모두 ON DELETE RESTRICT, ON UPDATE CASCADE로 설정했습니다. vehicle과 sale이 one-to-one이므로 VIN에 UNIQUE 제약을 부여해서 한 차량이 두 번 판매되는 상황을 차단했습니다.
- contracts: (supplier_id, model_id, part_type)을 기본키로 설정하여 다중값 속성 part_type을 원자적으로 저장했습니다. supplier_id와 model_id는 각각 supplier, model을 참조하는 외래키이며, 공급업체나 모델 삭제 시 관련 계약도 함께 정리되도록 ON DELETE CASCADE, ON UPDATE CASCADE로 설정했습니다.
- 쿼리에서 자주 사용되는 항목들인 sale(sale_date), model(body_style), vehicle(arrival_date), part(part_mfg_date)에 대해 보조 인덱스를 생성했습니다.
- trg_sale_set_status 트리거를 설정해 sale에 새 판매가 입력되면 해당 차량의 status를 자동으로 'Sold'로 갱신하도록 구현했습니다.

2) Sample Data Description

- 실제 자동차 회사 운영을 반영하도록 구성했습니다. brand는 'Hyundai', 'Tesla' 등의 10개 브랜드, model은 다양한 종류의 14개 모델로 설정했습니다. 판매 기록은 2023년부터 2026년까지로, 다양한 성별과 소득 구간을 보이게 설정했습니다.
- 모든 외래키가 존재하는 부모를 참조할 수 있도록 부모 테이블부터 차례대로 구성했습니다. part의 supplier_part_id, company_plant_id 둘 중 하나만 값을 가지게 하는 CHECK제약, sale의 VIN UNIQUE와 같은 조건들을 모두 만족하도록 데이터를 구성했습니다.
- Hyundai, Toyota에 최근 1년 판매를 집중시켜 판매 상위 브랜드 관련 쿼

리, 컨버터블 판매를 여름철에 배치해 계절성 관련 쿼리, 딜러별 재고 보유 기간을 다르게 설정해 재고 효율성 쿼리, 특정 공급업체의 변속기와의 다수 모델 계약을 통해 불량 부품 추적과 공급사 커버리지 쿼리를 각각 효과적으로 검증할 수 있도록 데이터를 구성했습니다.

4. Application Development

1) Database Connectivity Implementation Details

MySQL C API를 사용하여 데이터베이스에 연결하고, 접속에 필요한 사용자, 비밀번호 등의 정보는 database.h로 분리했습니다. dbConnect 함수에서 mysql_init()과 mysql_real_connect()로 연결을 수행합니다. 연결에 성공하면 conn에 저장된 MYSQL 핸들을 main을 포함한 함수들에서 사용합니다.

연결이 실패한 경우 mysql_error()로 에러 메시지를 출력하고, 프로그램을 종료하도록 구현했습니다.

2) Query Implementation

7개의 쿼리를 각각 별도의 함수로 구현하고, 공통 헬퍼 함수인 runQuery를 통해 실행되도록 했습니다. runQuery는 prepared statement 방식으로 동작하도록 구현했습니다. 쿼리에 기간과 관련된 조건은 DATE_SUB(CURDATE(), INTERVAL 1 YEAR)을 사용해 현재 날짜를 기준으로 계산되도록 했습니다.

- Sales Trends

입력 없이 지난 3년간의 판매를 대상으로 합니다. sale, customer, vehicle, model, brand를 join해서 브랜드, 연도, 월, 주, 구매자 성별, 소득 구간(3구간)으로 그룹화해서 그룹마다 판매 대수 COUNT와 매출액 SUM을 출력합니다.

- Defective Part Tracking

부품명, 공급업체 id, 제조 시작일, 종료일을 입력받습니다. part, supplier_plant, vehicle, model, sale, customer을 join해 해당 공급업체가 해당 기간동안 만든 부품을 찾습니다. 부품이 장착된 차량의 VIN, 모델명, 구매한 고객명과 판매일을 출력합니다.

- Top Brands by Revenue

입력 없이 지난 1년을 대상으로 합니다. sale, vehicle, model, brand를 join해 브랜드별 매출액 SUM을 구하고, 내림차순 정렬을 통해 상위 2개

브랜드를 출력합니다.

- Top Brands by Unit Sales
입력 없이 지난 1년을 대상으로 합니다. sale, vehicle, model, brand를 join해 브랜드별 판매 대수 COUNT를 구하고, 상위 2개 브랜드를 출력합니다.
- Seasonal Sales Patterns
차체 스타일을 입력받습니다. sale, vehicle, model을 join하여 해당 차체 스타일의 판매를 월별로 그룹화합니다. 판매가 많은 달 순으로 정렬하여 출력합니다.
- Dealer Inventory Efficiency
sale, vehicle, dealer를 join하고, arrival_date가 있는 차량을 대상으로 입고일부터 판매일까지의 일수를 DATEDIFF로 계산하여 딜러별 평균 재고 보유 일수를 AVG로 계산합니다. 평균이 큰 딜러 순으로 정렬해 출력합니다.
- Supplier Coverage
contracts와 supplier를 join하여 공급업체별로 부품을 공급하는 서로 다른 모델 수를 COUNT DISTINCT model_id를 통해 구합니다. 내림차순으로 정렬해 출력합니다.

3) Error Handling and User Interface Design

사용자 인터페이스는 명세서처럼 메뉴 기반의 switch case로 구현했습니다. 0이 입력되면 반복을 종료하고, 1~7의 값이 입력되면 해당 쿼리 함수를 호출합니다.

readLine()과 readInt() 함수를 통해서 사용자 입력을 처리했습니다. 숫자 입력이 비어 있으면 def를 사용할 수 있도록 구현했습니다. 날짜 입력은 validDate로 'YYYY-MM-DD' 형식을 확인하고, 올바르지 않은 경우 에러 메시지를 출력하도록 했습니다.

쿼리 실행 단계에서 mysql_stmt_prepare()과 mysql_stmt_execute()의 반환값을 검사하여 오류가 발생하면 오류 메시지를 출력하고 쿼리를 중단할 수 있도록 구현했습니다.

5. Testing and Validation

1) Sales Trends

```
Select [0-7]: 1
brand_name | yr | mon | wk | gender | income_range | units | revenue
BMW | 2023 | 7 | 27 | M | High(>=100k) | 1 | 59400.00
BMW | 2025 | 7 | 30 | M | Low(<50k) | 1 | 59400.00
Ford | 2024 | 8 | 30 | M | Low(<50k) | 1 | 49820.00
Honda | 2023 | 11 | 46 | F | Mid(50k-100k) | 1 | 24000.00
Honda | 2025 | 9 | 37 | M | High(>=100k) | 1 | 31000.00
Hyundai | 2024 | 1 | 2 | M | Mid(50k-100k) | 1 | 28000.00
Hyundai | 2024 | 2 | 8 | F | Mid(50k-100k) | 1 | 22000.00
Hyundai | 2025 | 3 | 10 | M | Mid(50k-100k) | 1 | 22000.00
Hyundai | 2025 | 7 | 26 | F | Mid(50k-100k) | 1 | 28000.00
Hyundai | 2025 | 11 | 44 | F | Low(<50k) | 1 | 22000.00
Hyundai | 2026 | 1 | 2 | M | Low(<50k) | 1 | 22000.00
Hyundai | 2026 | 3 | 10 | F | Mid(50k-100k) | 1 | 28000.00
Hyundai | 2026 | 5 | 18 | M | High(>=100k) | 1 | 22000.00
Kia | 2024 | 5 | 20 | F | High(>=100k) | 1 | 30000.00
Kia | 2025 | 5 | 18 | F | Mid(50k-100k) | 1 | 26000.00
Nissan | 2025 | 12 | 49 | F | High(>=100k) | 1 | 26000.00
Tesla | 2024 | 10 | 40 | M | Mid(50k-100k) | 1 | 42000.00
Toyota | 2023 | 7 | 29 | F | High(>=100k) | 1 | 52500.00
Toyota | 2023 | 9 | 36 | M | Low(<50k) | 1 | 27000.00
Toyota | 2024 | 6 | 23 | F | Mid(50k-100k) | 1 | 27000.00
Toyota | 2024 | 8 | 32 | M | High(>=100k) | 1 | 53500.00
Toyota | 2025 | 8 | 33 | F | High(>=100k) | 1 | 27000.00
Toyota | 2025 | 10 | 42 | M | Mid(50k-100k) | 1 | 27000.00
Toyota | 2026 | 2 | 7 | M | Mid(50k-100k) | 1 | 27000.00
Toyota | 2026 | 4 | 14 | F | Mid(50k-100k) | 1 | 52500.00
Toyota | 2026 | 5 | 21 | F | Low(<50k) | 1 | 27000.00
Volkswagen | 2024 | 11 | 47 | F | Low(<50k) | 1 | 29000.00
```

지난 3년의 판매를 브랜드, 연도, 월, 주, 성별, 소득 구간별로 그룹을 만들고 해당 그룹의 정보와 판매 대수, 매출액을 출력합니다.

2) Defective Part Tracking

```
Select [0-7]: 2
Part name (e.g., Transmission / Engine / Battery): Transmission
Supplier id: 1
Start date YYYY-MM-DD (e.g., 2024-01-01): 2024-01-01
End date YYYY-MM-DD (e.g., 2024-06-30): 2025-12-31
part_id | part_name | VIN | model_name | customer_name | sale_date
1 | Transmission | VIN000000000000008 | Tucson | Lim Jisung | 2024-01-18
2 | Transmission | VIN000000000000009 | Sportage | Han Yuna | 2024-05-22
3 | Transmission | VIN000000000000010 | Mustang | Oh Minjae | 2024-08-03
4 | Transmission | VIN000000000000011 | Camry | Seo Arin | 2024-06-14
5 | Transmission | VIN000000000000012 | Model 3 | Shin Taeyang | 2024-10-09
```

부품명-Transmission, 공급업체-1, 2024-01-01~2025-12-31을 기간으로 입력한 결과 해당 공급업체가 해당 기간에 만든 변속기 5개와, 장착 차량의 VIN, 모델명, 구매 고객명, 판매일을 출력합니다.

3) Top Brands by Revenue

```
Select [0-7]: 3
brand_name | revenue
Toyota | 160500.00
Hyundai | 122000.00
```

지난 1년 동안의 매출액 상위 2개 브랜드를 매출액과 함께 출력합니다.

4) Top Brands by Unit Sales

```
Select [0-7]: 4
brand_name | units
Hyundai | 5
Toyota | 5
```

지난 1년 동안의 판매 대수 상위 2개 브랜드를 판매 대수와 함께 출력합니다.

5) Seasonal Sales Patterns

```
Select [0-7]: 5
Body style (e.g., Sedan / SUV / Convertible): Convertible
month | units
7 | 3
8 | 2
4 | 1
```

차체 스타일-Convertible을 입력한 결과로 해당 차체 스타일의 차량이 판매된 달과 판매 대수를 내림차순으로 출력합니다.

6) Dealer Inventory Efficiency

```
Select [0-7]: 6
dealer_id | dealer_name | avg_days
7 | Suwon Sales | 190.0
6 | Gwangju Garage | 170.0
4 | Daegu Wheels | 85.0
3 | Incheon Cars | 70.0
5 | Daejeon Drive | 60.0
2 | Busan Auto | 35.0
1 | Gangnam Motors | 25.0
```

입고일부터 판매일까지 평균 보유 일수가 가장 긴 달러 순으로 출력합니다.

7) Supplier Coverage

```
Select [0-7]: 7
supplier_id | supplier_name | model_count
1 | Hyundai Mobis | 10
2 | Bosch | 3
3 | Denso | 2
9 | Hanon Systems | 2
5 | Continental | 1
6 | Aisin | 1
```

서로 다른 모델에 공급하는 부품이 많은 공급업체 순으로 출력합니다.

- 잘못된 데이터 삽입을 통해 제약과 트리거가 규칙 위반을 확인하는지 검증했습니다. part의 두 공장 값을 둘 다 채우거나 둘 다 비우면 CHECK (chk_part_one_plant)를 통해 거부를 확인했습니다. 판매된 VIN이 다시 판매로 들어오면 UNIQUE를 통해 거부를 확인했습니다. gender와 sale_price등 쿼리에 필요한 필수 값 누락 및 음수 금액은 NOT NULL, CHECK를 통해 확인했습니다. 또한 판매 삽입 시 해당 차량의 status가 자동으로 'Sold'로 변경됩니다.